COMS 4130 Project Status Update 2

by Abby Van Der Brink, Taylor Bauer, and Trevor List

COMPLETED

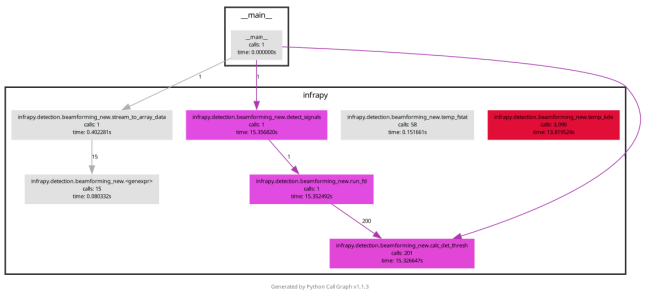
CFG & Call Graph (Taylor)

- analysis focused on infrapy/detection/beamforming_new.py
- changed from initial plan of using py2cfg for creating cfg to using AST because py2cfg was having issues/failing to parse beamforming_new.py because of some compatibility issues with Numpy and other libs and I could not find a fix
    - it reads the source code without needing to execute it like py2cfg does
- two scripts were created for the change
    - call_struct.py for going through each function call and its relationship and creates json of the pipeline for the database to use
    - cfg.py goes through four functions we have defined as like the core logic and creates a cfg for each of these functions and its logic as well as outputs json for these and visualizations
        - run_fd: Adaptive fisher detection function and has the main detection logic
        - calc_det_thresh: this calculates the detection threshold
        - run_fk: this is the entry point for the beamforming pipeline where signal processing is happening before detection
        - find_peaks: this finds the signal peaks in the beamforming output
- **KNOWN ISSUE:** currently resolving a graph rendering issue with graphviz that is giving the CFG's a horizontal output but the data and json are correct
- files given to abby for database queries
    - call.json - call graph edges
    - cfg.json - cfg block edges
- radon: it's used for determining complexity, basically it just counts how many branches are in a function and then gives a grade like A for less complex and D for more complex

and it was ran on beamforming_new.py

```
F 97:0 fft_array_data - E (32)
F 517:0 run - D (30)
F 969:0 run_fd - D (23)
F 427:0 compute_beam_power - C (14)
F 636:0 find_peaks - C (12)
F 31:0 stream_to_array_data - C (11)
F 280:0 compute_delays - B (6)
F 884:0 run_fk - B (6)
F 356:0 project_ABA - A (4)
F 388:0 project_ABc - A (4)
F 323:0 project_Ab - A (3)
F 606:0 pure_state_filter - A (3)
F 755:0 project_beam - A (3)
F 801:0 extract_signal - A (2)
F 848:0 calc_det_thresh - A (2)
F 253:0 build_slowness - A (1)
F 513:0 compute_beam_power_wrapper - A (1)
F 873:0 beam_window - A (1)
F 880:0 beam_window_wrapper - A (1)
F 1129:0 detect_signals - A (1)
```

- full call graph



Database Populated (Abby)

- Used the data given by Taylor in cfg.json and call.json to populate the graph
- I made a python file named populate_db.py to do this

```
import sqlite3, json

DB = "413db.db"

with open("call.json") as f:
    call_data = json.load(f)

with open("cfg.json") as f:
    cfg_data = json.load(f)

conn = sqlite3.connect(DB)

conn.executemany("""
    INSERT INTO static_analysis (analysis_type, caller, callee,
file, line_number, extra_data)
```

```
    VALUES (:analysis_type, :caller, :callee, :file, :line_number,
:extra_data)
""", call_data["static_analysis_rows"])

conn.executemany("""
INSERT INTO static_analysis (analysis_type, caller, callee, file,
line_number, extra_data)
    VALUES (:analysis_type, :caller, :callee, :file, :line_number,
:extra_data)
""", cfg_data["static_analysis_rows"])

conn.executemany("""
 INSERT INTO metadata (function_name, file, input_range_notes)
    VALUES (:function_name, :file, :input_range_notes)
""", [
    {
        "function_name": r["function_name"],
        "file": r["file"],
        "input_range_notes": f"lines
{r['start_line']}-{r['end_line']} ({r['line_count']} lines)"
    }
    for r in call_data["metadata_rows"]
])

conn.commit()
conn.close()

print(f"Inserted {len(call_data['static_analysis_rows'])} call graph
edges")
print(f"Inserted {len(cfg_data['static_analysis_rows'])} CFG edges")
print(f"Inserted {len(call_data['metadata_rows'])} metadata rows")
```

- Here is the result of running the file:
  >python populate_db.py
  >Inserted 16 call graph edges
  >Inserted 127 CFG edges
  >Inserted 22 metadata rows

Query Module - Load Call Graph (Abby)

- To start the call graph query module, I created a function that loads the existing call graph from the database
- This function will be used in the later queries written

```
# load graph from DB
def load_call_graph(db_path: str) -> tuple:
    conn = sqlite3.connect(db_path)

    rows = conn.execute("""
        SELECT caller, callee
        FROM static_analysis
        WHERE analysis_type = 'call_graph'
    """).fetchall()
```

```
    conn.close()

    forward = collections.defaultdict(list)
    reverse = collections.defaultdict(list)
    for caller, callee in rows:
        forward[caller].append(callee)
        reverse[callee].append(caller)

    return dict(forward), dict(reverse)
```

- Also, decided on what the supported queries will be
    - is_caller(A, B) - checks if A is a direct/indirect caller of B
    - all_callees(A) - checks what functions A can reach
    - all_callers(B) - checks what functions will eventually call B
    - shortest_path(A, B) - shortest path from A to B
    - direct_callees(A) - checks what A calls directly
    - direct_callers(B) - checks what calls B directly
    - These queries will be implemented in the call_graph_query.py file with the load_call_graph function

Abstract Interpretation & Dataflow (Trevor)

- Used call graph results to reconstruct the full detection pipeline
    - Stream_to_array_data → run_fk → beam_window → compute_beam_power → find_peaks → run_fd
- Identified key variables across stages
    - Beam_power (signal energy)
    - Peak_values (detected maxima)
    - Det_threst (detection threshold)
    - Det_stat (detection statistic)
- Defined input constraintspeak)
    - Beam_power, peak_values ≥ 0
    - Det_thresh >0
    - Array_data must be non-empty real-valued time series
- Traced dataflow dependencies:
    - Beam_power flows from compute_beam_power → find_peaks → run_fd
    - Peak_values influence detection decisions in run_fd
    - Det_thresh is computed separately and used for comparison
- Developed abstract interpretation rules
    - Detection occurs when det_stat > det_thresh
    - High noise raise det_thresh → risk of false negatives
    - Low noise lowers det_thresh → risk of false positives
- Identified failure scenarios

    - Incorrect peak detection leads to missed or false detections
    - Boundary conditions near threshold create unstable classifications
    - Upstream errors in beamforming propagate to incorrect detection outcomes

Some Future Work/Next Steps For Final Project

Taylor

- Fix CFG image rendering (by 4/5)
- Dataflow Analysis (by 4/7)
    - to do this i can either create another script using ast and create a json file or i can do this manually, might possibly look into other tools and this will be done on the functions we are interested in

Abby

- Finish the call_graph_query file including an interactive shell for usability (4/6-4/8)
- Test query file against the actual call.json data in the database (4/8-4/9)
- Add SQL coverage queries (4/10)

Trevor

- Refine abstract interpretation rules (by 4/7)
- Identity input ranges for key variables and consider storing them in the database (by 4/7)
- Trace variable propagation through pipeline (tie into dataflow work) (4/7-4/9)
- fuzzing environment using sample data and exploring tools like pyfuzz (by 4/10)
- Plan fuzzing test for edge cases (noise, weak signals, threshold boundaries) (by 4/10)